Lab 2: Hands-On Git and Version Control

Lab2_Solution

Name: Yusra Faisal

(Bcs-vii b)

099

GitHub username: faisal-yusra

Course: CS-649 Cloud DevOps

Environment: Ubuntu Server 26.04.1 LTS on VMware Workstation, Git 2.53.0

Practice repository: DevOps_Lab2_Practice (private)

Submission: DevOps / Labs / Lab02

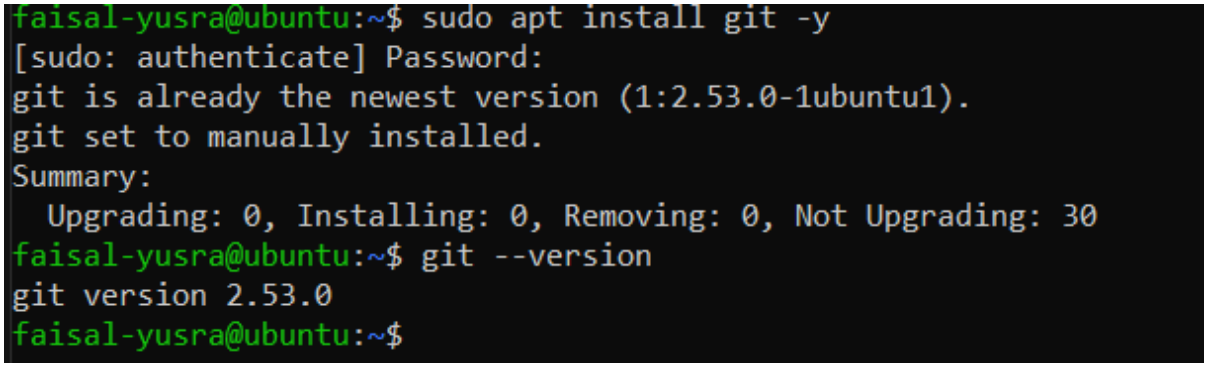
Getting Started: Install Git

Git was already installed on the Ubuntu VM; the install command confirms the newest version (2.53.0).

Commands

```
sudo apt install git -y
git --version
```

Evidence



git_installation.png

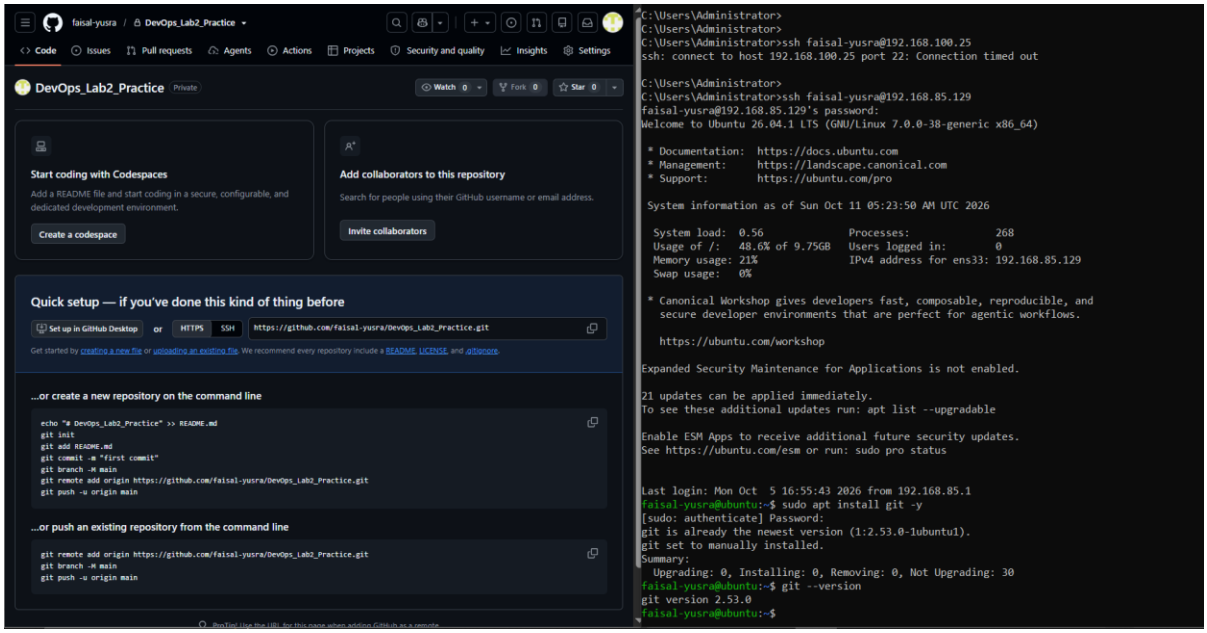
Task 1: Create Private GitHub Repository

A private repository named DevOps_Lab2_Practice was created on GitHub without a README, so the first push is clean.

Commands

```
(GitHub web UI: New repository -> Private)
```

Evidence



repo_private.png

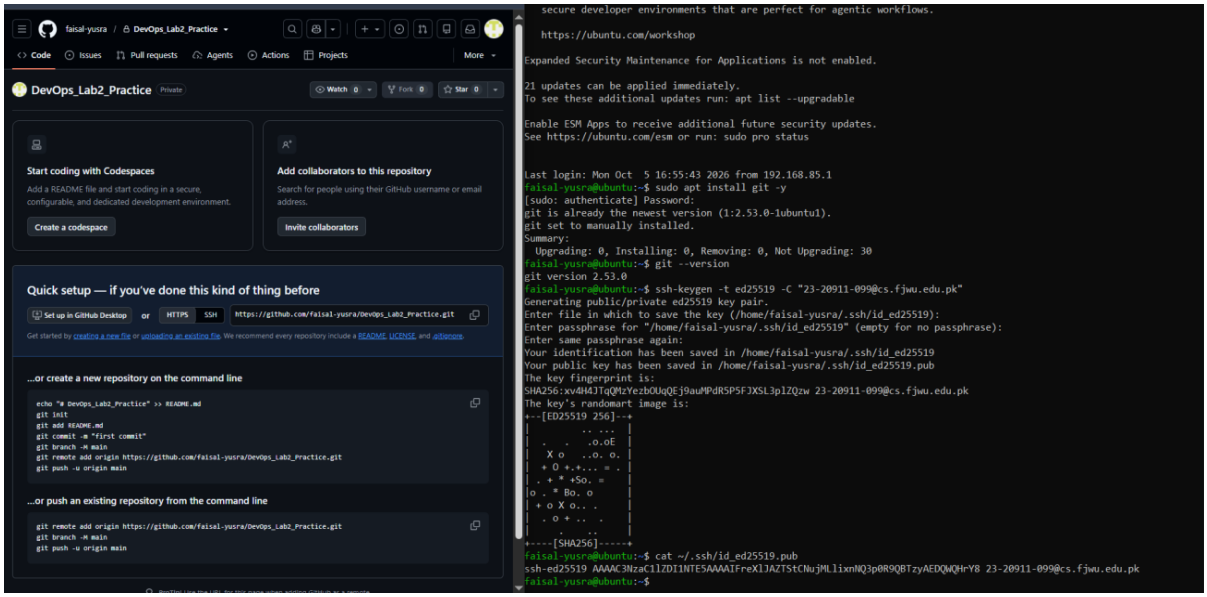
Task 2: Connect Repository via SSH

An ed25519 key was generated on the VM, the public key was added to GitHub, and the repository was cloned over SSH. Only the public key is shown.

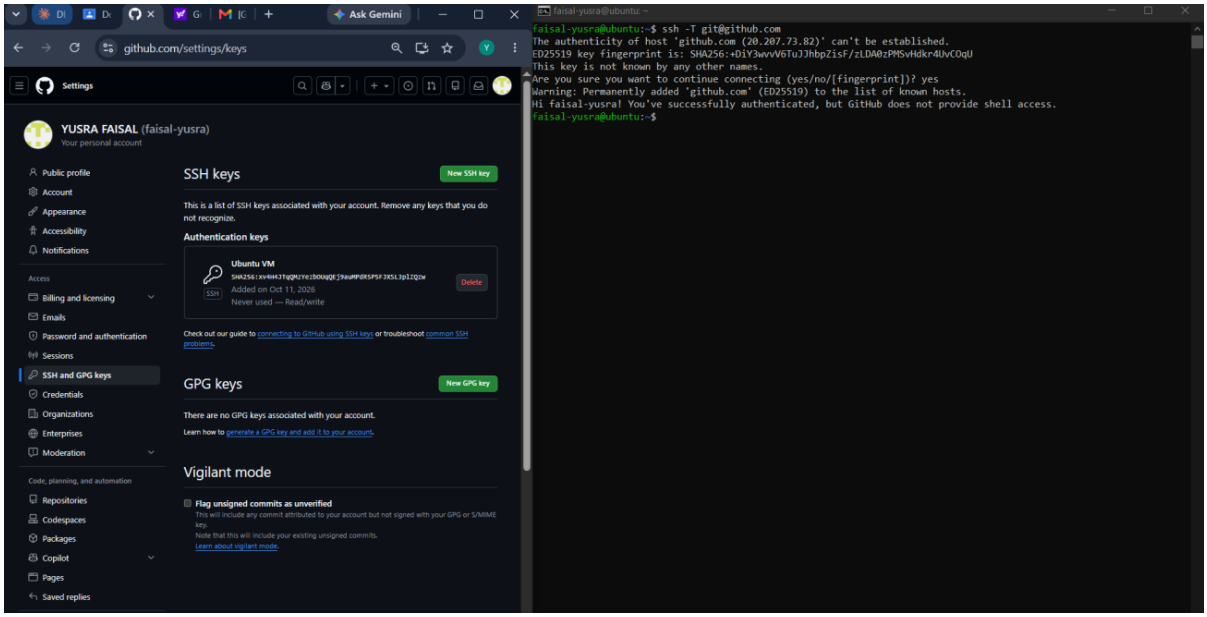
Commands

```
ssh-keygen -t ed25519 -C "<email>"
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
git clone git@github.com:faisal-yusra/DevOps_Lab2_Practice.git
```

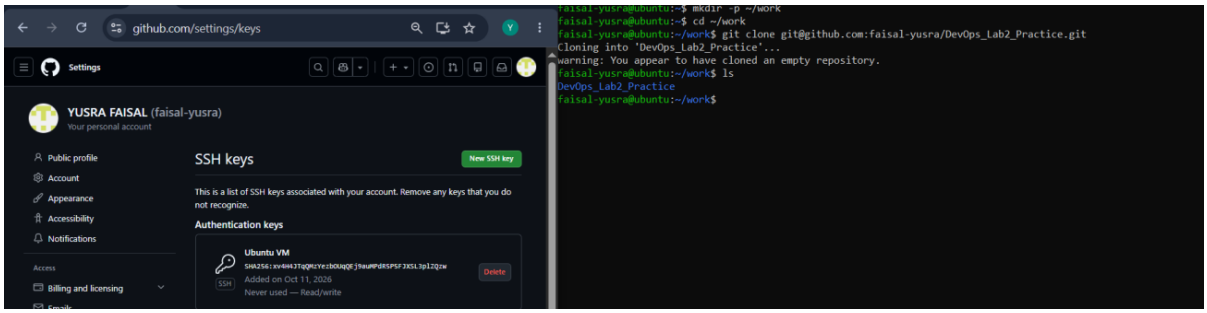
Evidence



ssh_keygen.png



github_sshkey.png



ssh_clone.png

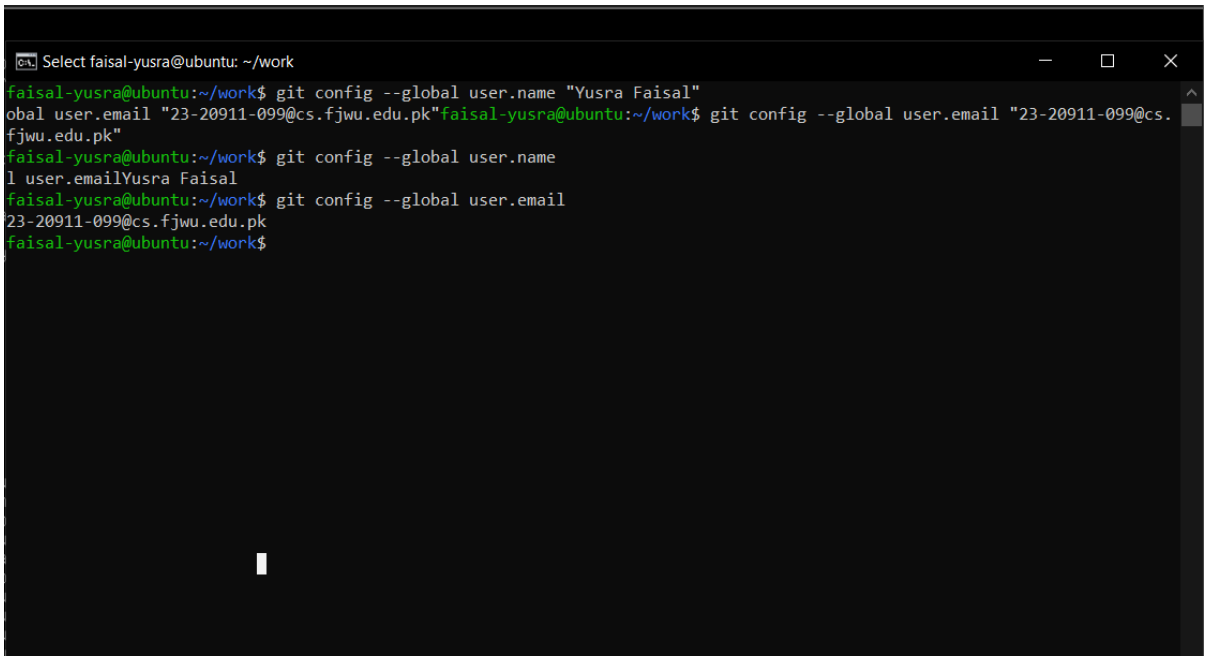
Task 3: Configure Git Username and Email

The identity used for all commits was configured and verified.

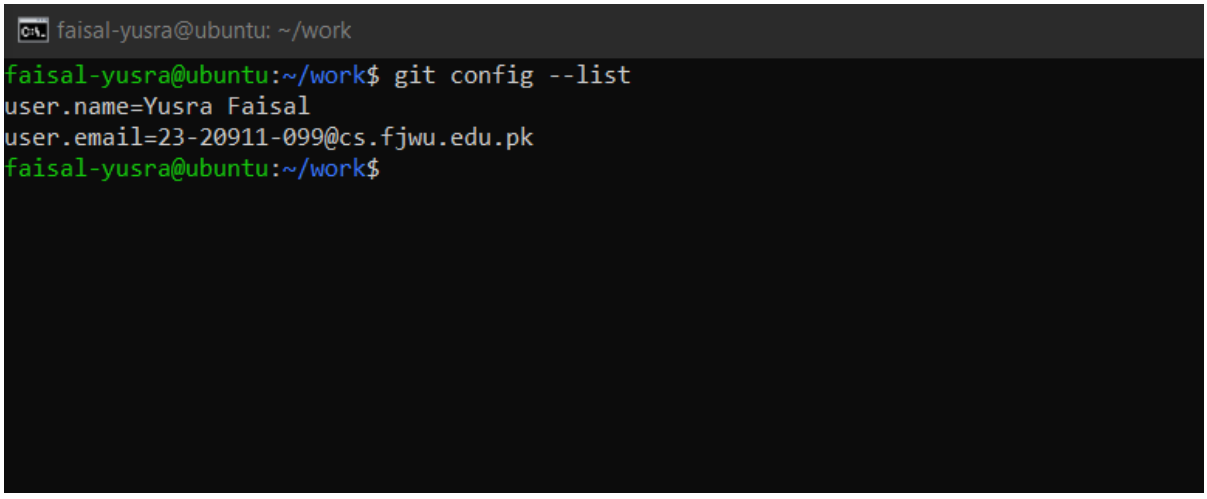
Commands

```
git config --global user.name "Yusra Faisal"
git config --global user.email "<email>"
git config --list
```

Evidence



git_identity.png



git_config_list.png

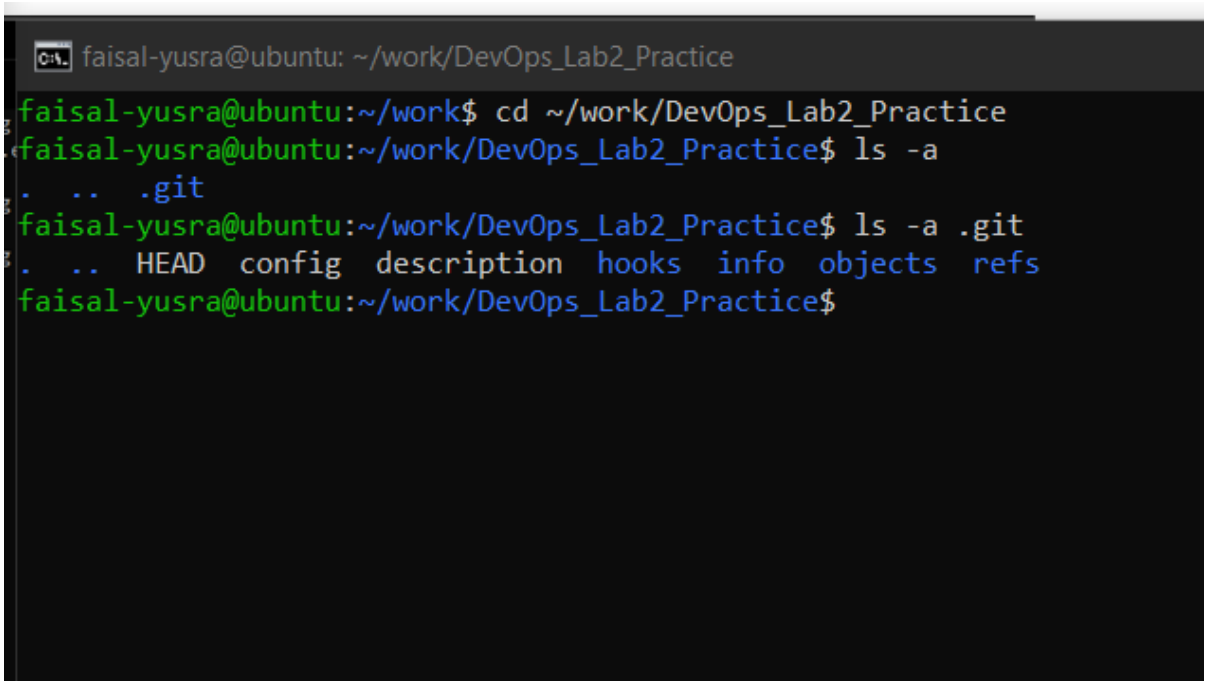
Task 4: Explore the .git Folder

Hidden files were listed to locate .git, then its contents (HEAD, config, hooks, objects, refs) were listed.

Commands

```
ls -a
ls -a .git
```

Evidence



git_folder.png

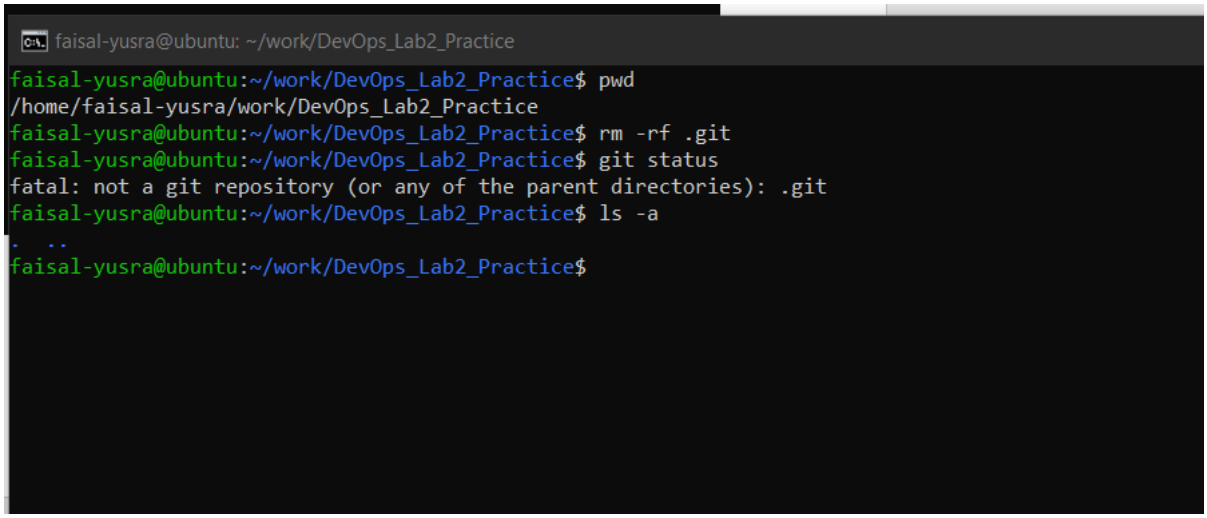
Task 5: Local Repository Management

The .git folder was deleted, which made git status fail. The repository was re-initialised, README.md was committed, and the work was pushed to GitHub. Why does git status fail? The .git folder is the repository: it holds history, branches, remotes and configuration. Without it the directory is an ordinary folder, so Git reports "not a git repository".

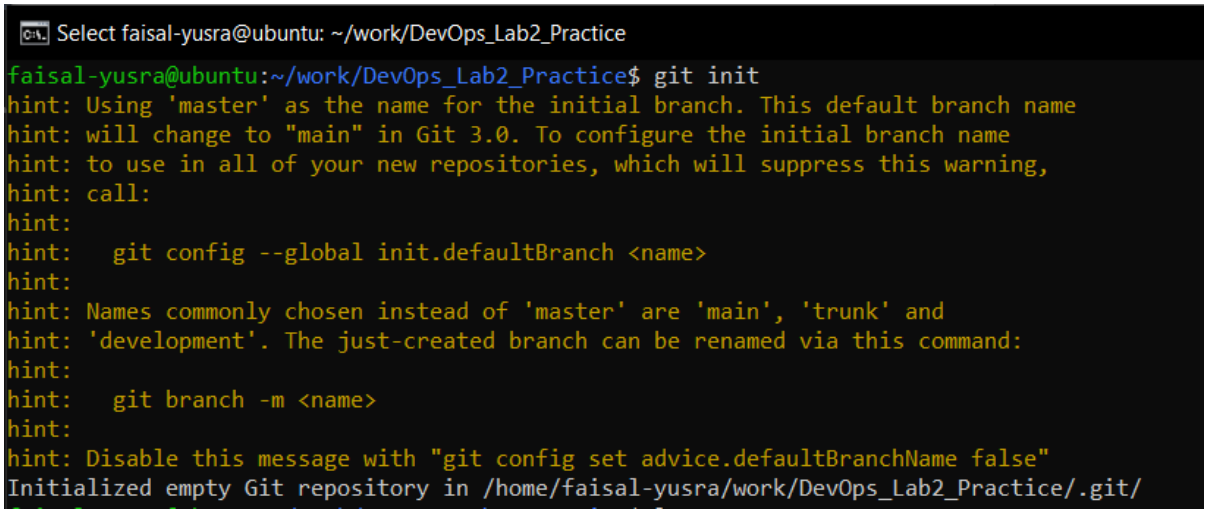
Commands

```
rm -rf .git
git status
git init
echo "# Lab02 Git Practice" >> README.md
git add README.md
git commit -m "Initial commit"
git remote add origin git@github.com:faisal-yusra/DevOps_Lab2_Practice.git
git branch -M main
git push -u origin main
```

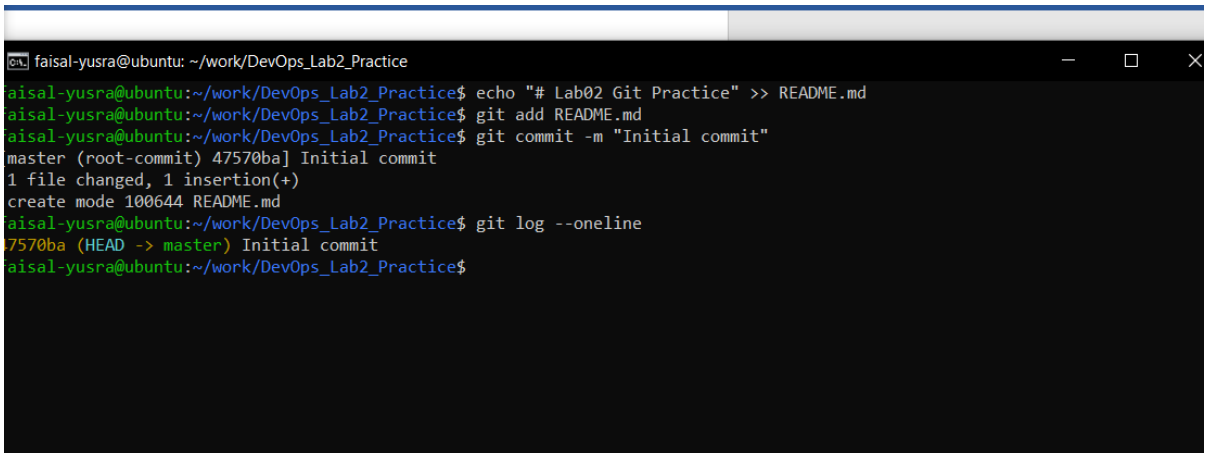
Evidence



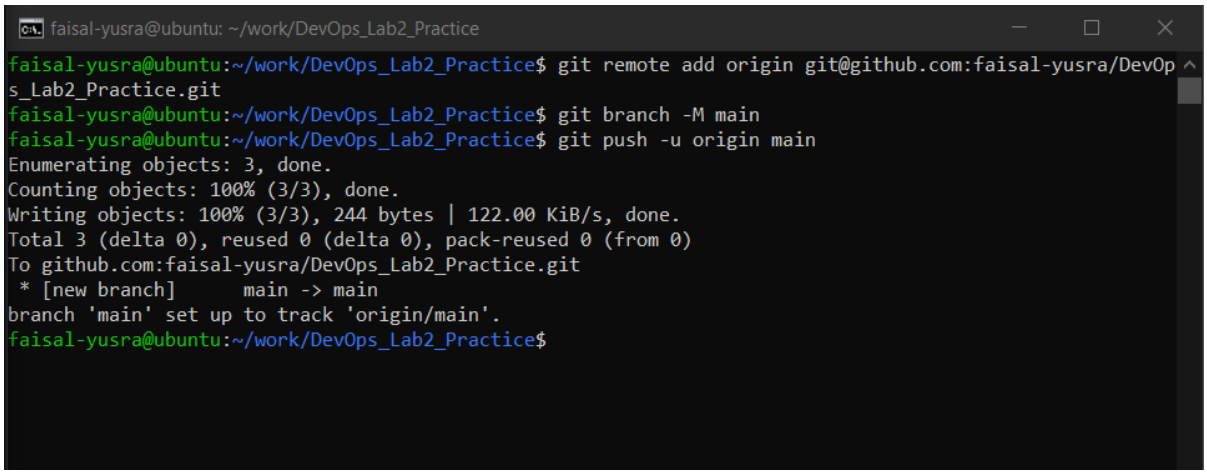
delete_git.png



git_init.png



first_commit.png



first_push.png

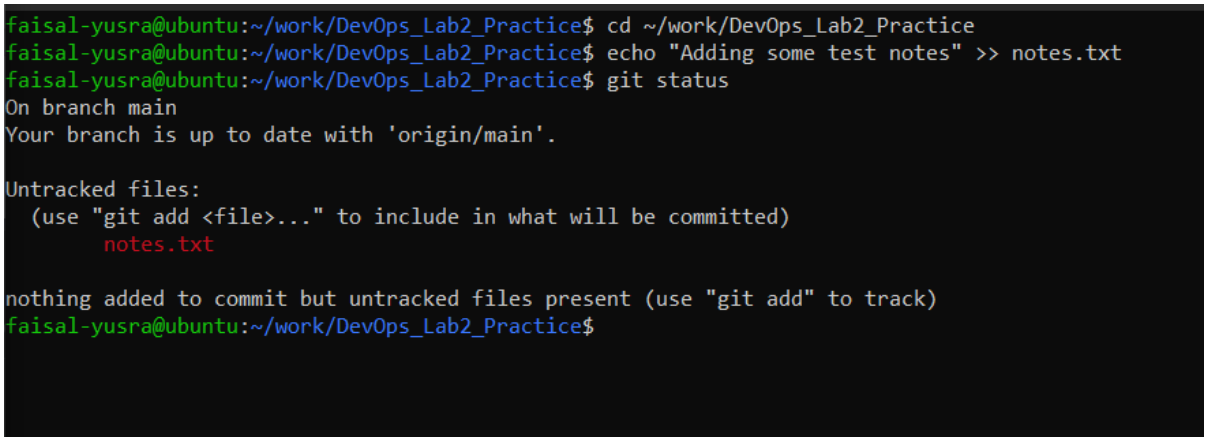
Task 6: File Status and Staging

notes.txt was created, its status checked, then staged, committed and later edited and pushed.

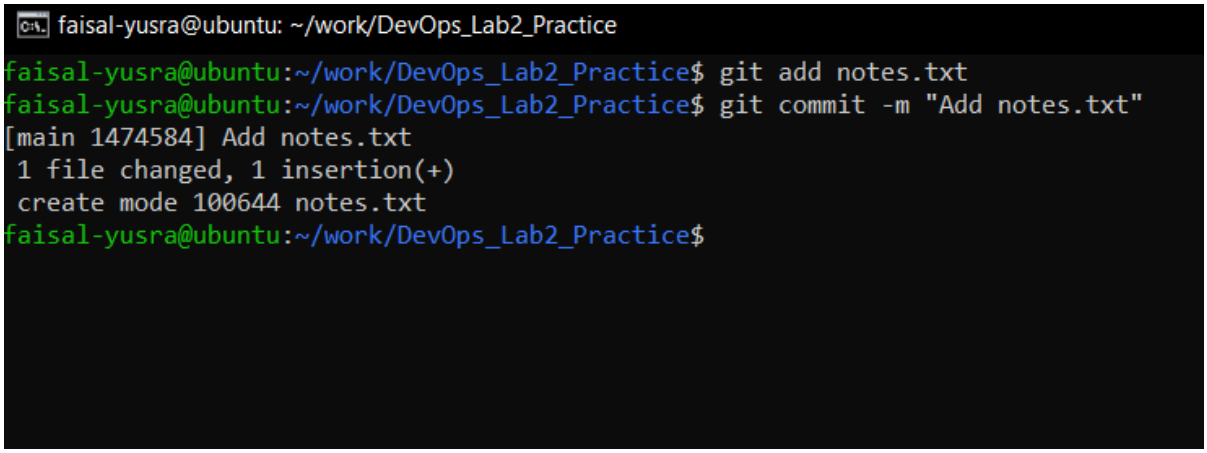
Commands

```
echo "Adding some test notes" >> notes.txt
git status
git add notes.txt
git commit -m "Add notes.txt"
```

Evidence



status1.png



commit_notes.png

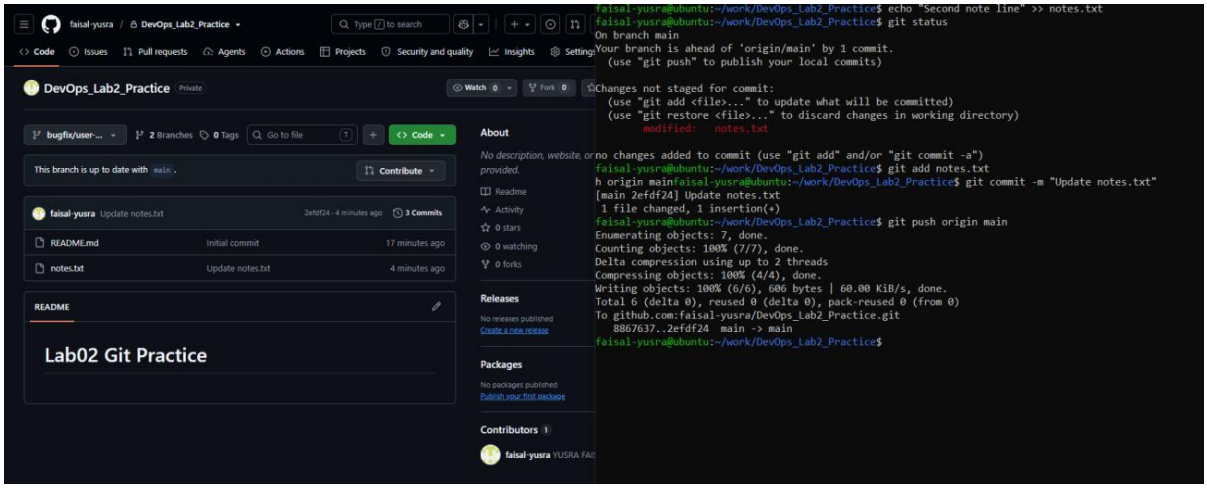
Task 7: Branch Creation Using GitHub GUI

The branch bugfix/user-auth-error was created on GitHub and then brought into the local repository.

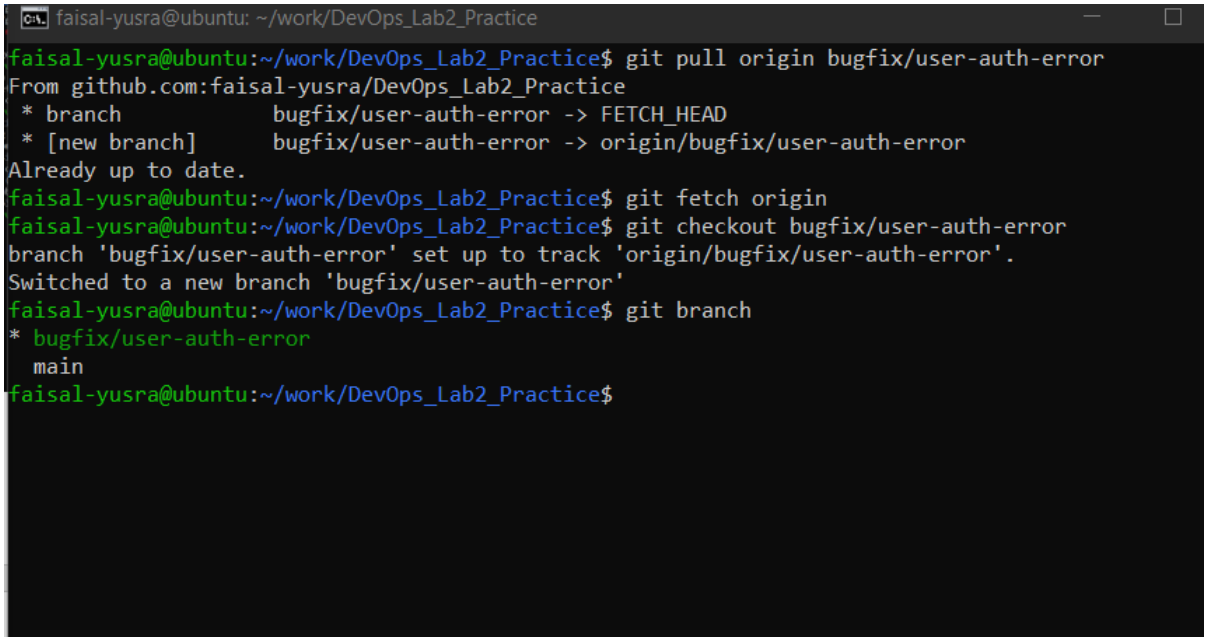
Commands

```
git pull origin bugfix/user-auth-error
git fetch origin
git checkout bugfix/user-auth-error
git branch
```

Evidence



bugfix_branch_gui.png



bugfix_branch_local.png

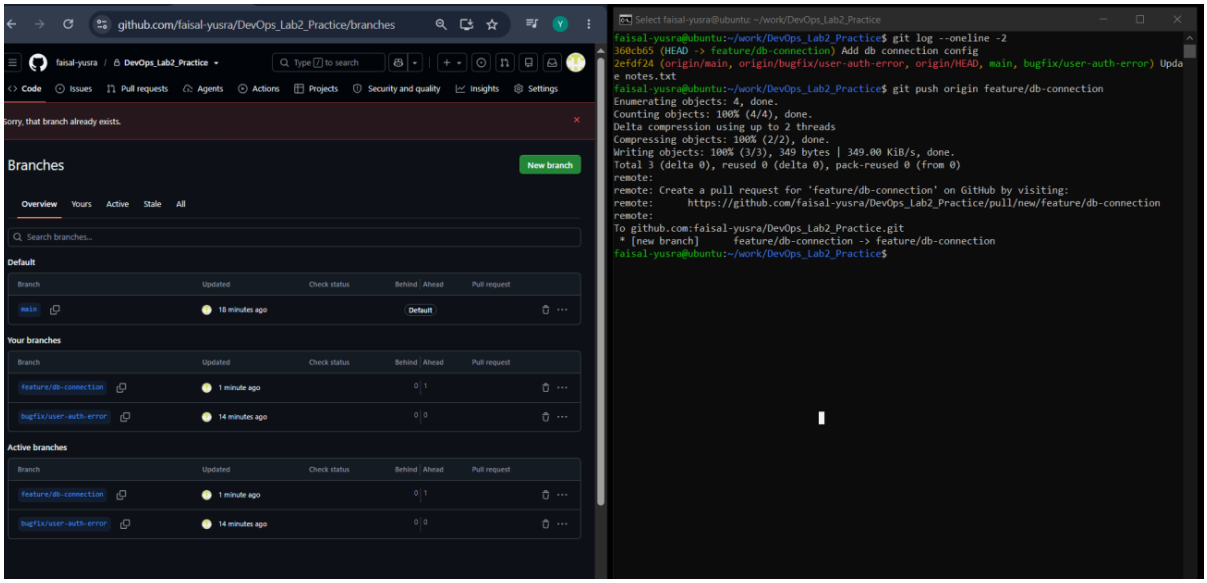
Task 8: Branch Creation and Push Using Git Bash

feature/db-connection was created in the terminal, a file was committed on it, and the branch was pushed.

Commands

```
git checkout -b feature/db-connection
echo "# DB connection config" > db_config.txt
git add db_config.txt
git commit -m "Add db connection config"
git push origin feature/db-connection
```

Evidence



feature_db_branch.png

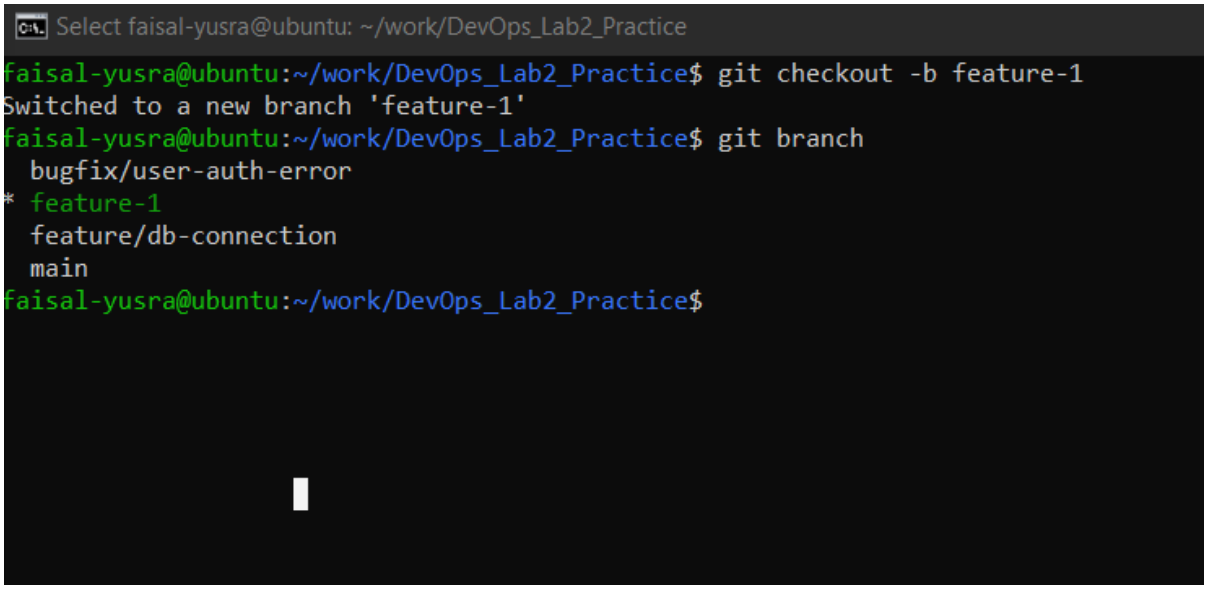
Task 9: Branching and Merging

A function was added to main.py on feature-1, merged into main, and both branches were pushed.

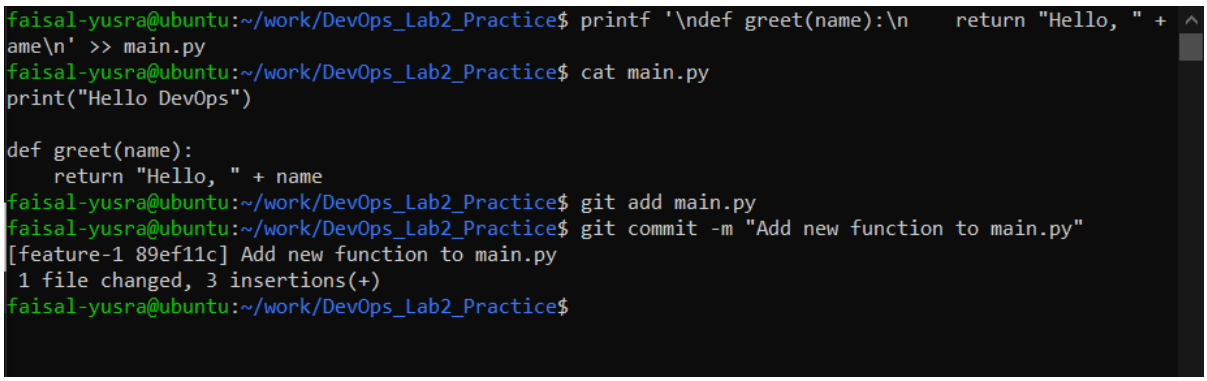
Commands

```
git checkout -b feature-1
git add main.py
git commit -m "Add new function to main.py"
git checkout main
git merge feature-1
git push origin main
git push origin feature-1
```

Evidence



branch_create.png



feature_commit.png

```
faisal-yusra@ubuntu: ~/work/DevOps_Lab2_Practice
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git merge feature-1
Updating 8ebd25a..89ef11c
Fast-forward
 main.py | 3 +++
 1 file changed, 3 insertions(+)
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$
```

merge.png

```
faisal-yusra@ubuntu: ~/work/DevOps_Lab2_Practice
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git push origin main

Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 389 bytes | 129.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:faisal-yusra/DevOps_Lab2_Practice.git
   8ebd25a..89ef11c  main -> main
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git push origin feature-1
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-1' on GitHub by visiting:
remote:      https://github.com/faisal-yusra/DevOps_Lab2_Practice/pull/new/feature-1
remote:
To github.com:faisal-yusra/DevOps_Lab2_Practice.git
 * [new branch]      feature-1 -> feature-1
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$
```

push_branches.png

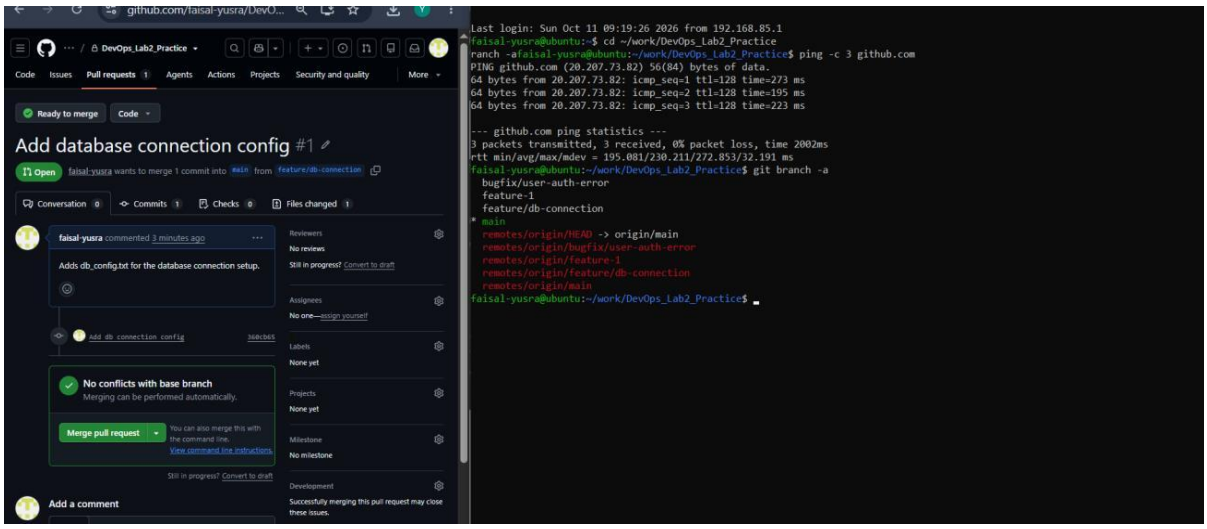
Task 10: Pull Request and Branch Review (GitHub GUI)

A pull request from feature/db-connection to main was created, reviewed in the Files changed tab, merged, and the branch was deleted.

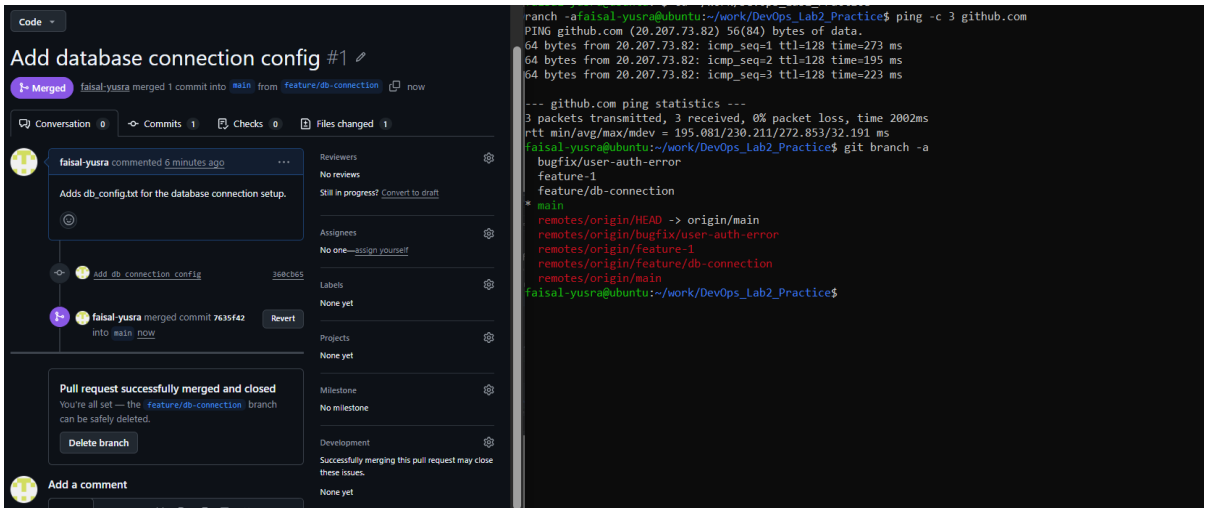
Commands

```
(GitHub web UI)
```

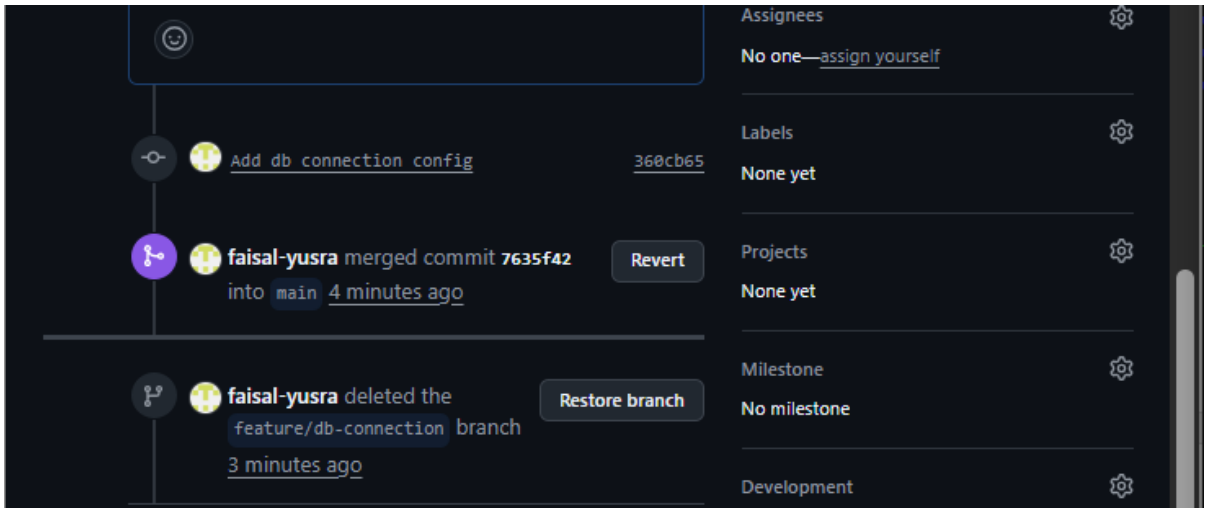
Evidence



pr_creation.png



pr_merge.png



branch_delete.png

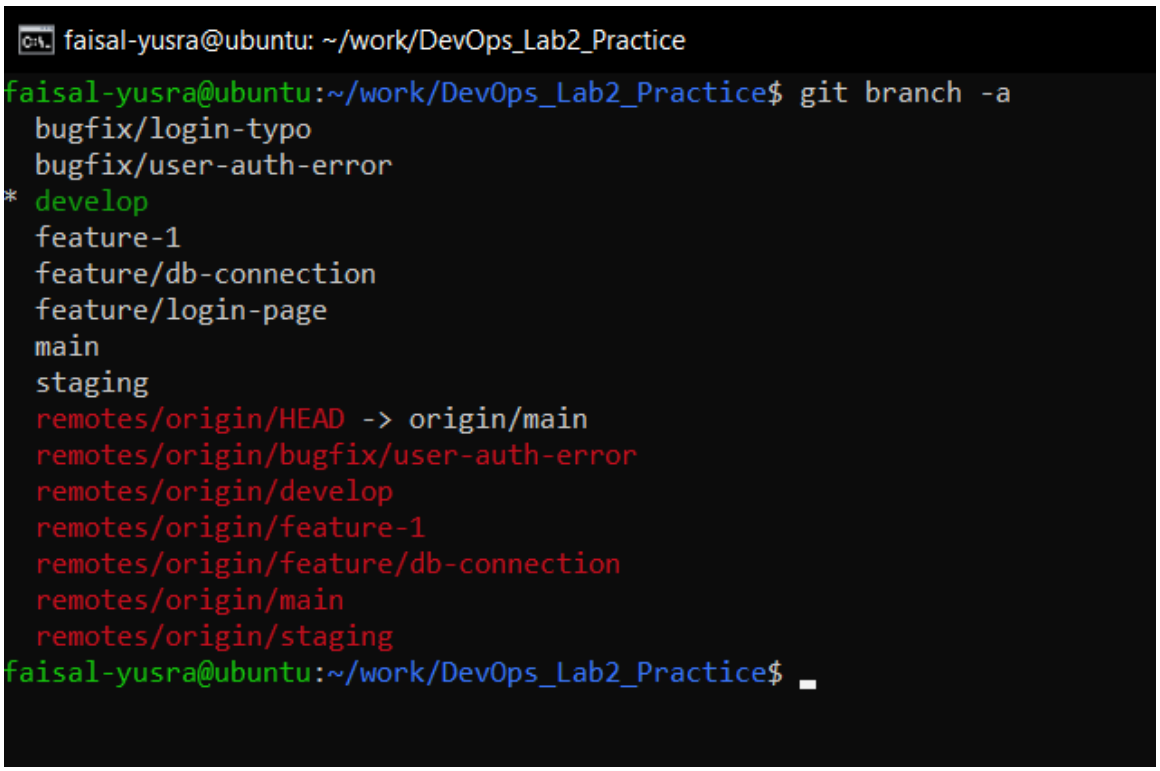
Task 11: Detailed Branch Strategy (Develop/Staging)

Flow used: feature/* and bugfix/* -> develop -> staging -> main, with --no-ff so each merge appears in history.

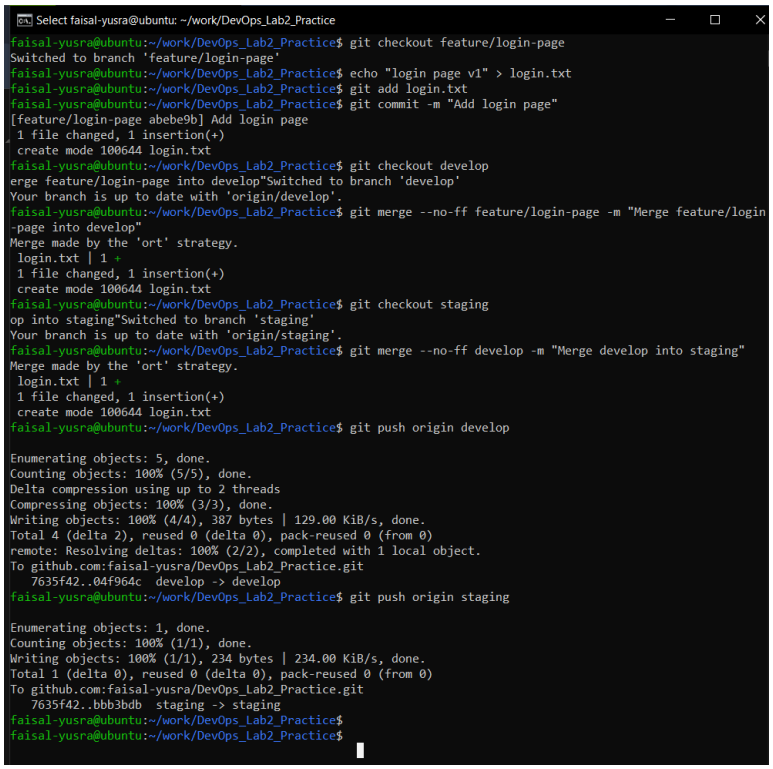
Commands

```
git checkout -b develop
git push -u origin develop
git checkout -b staging
git push -u origin staging
git merge --no-ff feature/login-page   (into develop)
git merge --no-ff develop   (into staging)
git merge --no-ff staging   (into main)
```

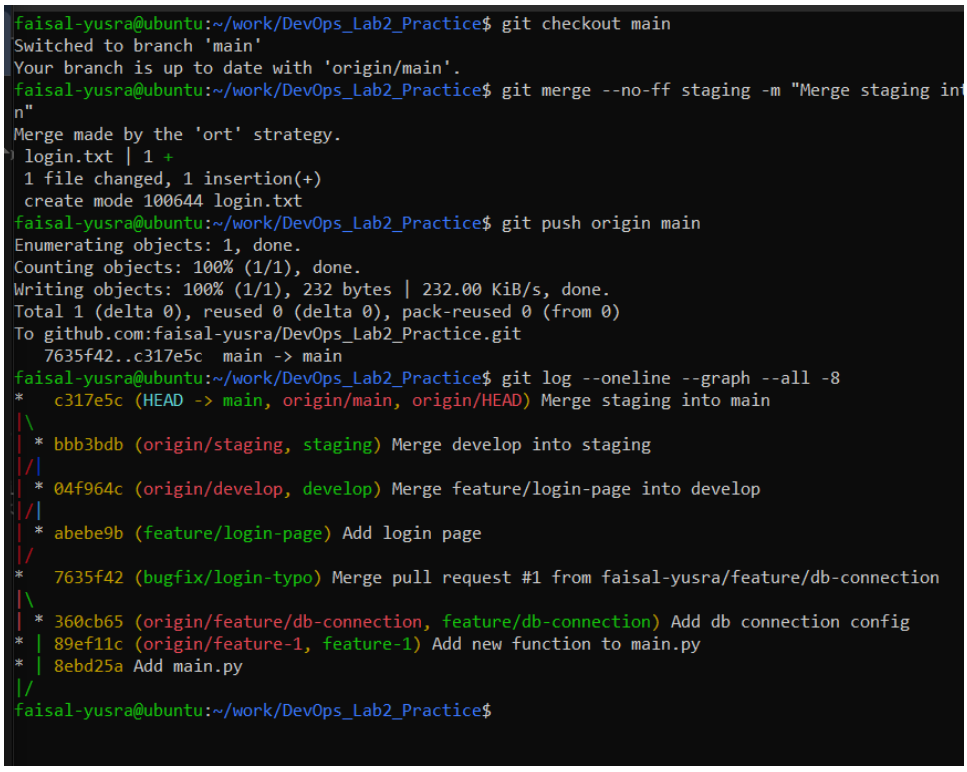
Evidence



branch_strategy.png



branch_merges.png



final_merge.png

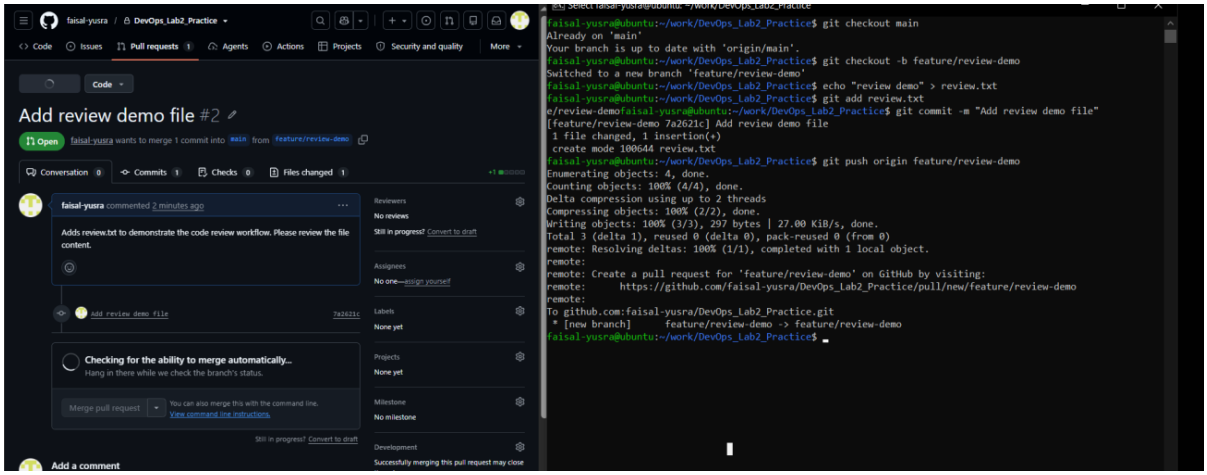
Task 12: Code Review Workflow

A second GitHub account acted as reviewer: reviewer assigned, changes requested, fixes pushed to the same branch (the PR updated automatically), then approved and merged. A separate practice PR was closed without merging.

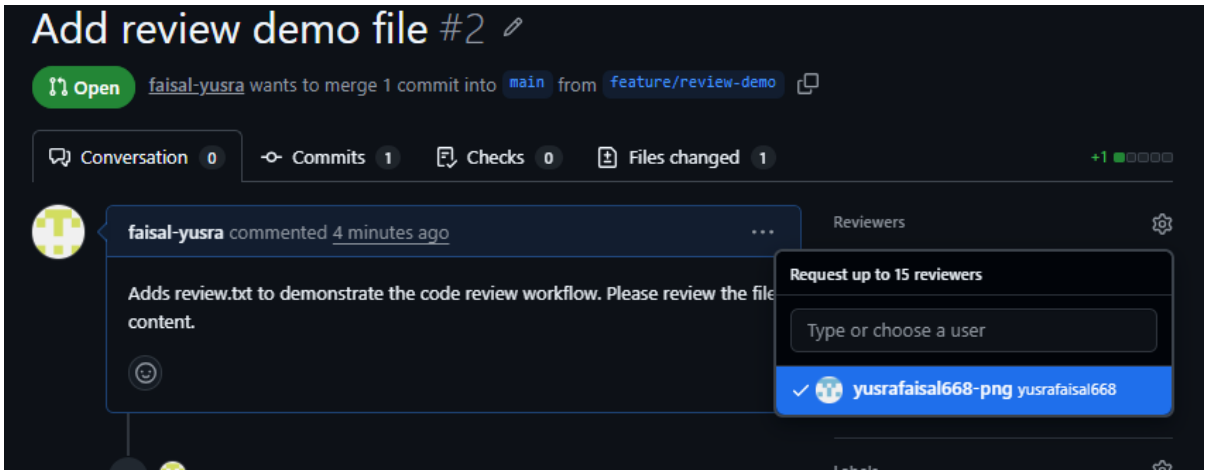
Commands

```
git checkout -b feature/review-demo
git push origin feature/review-demo
git commit -m "Address review comments"
git push origin feature/review-demo
```

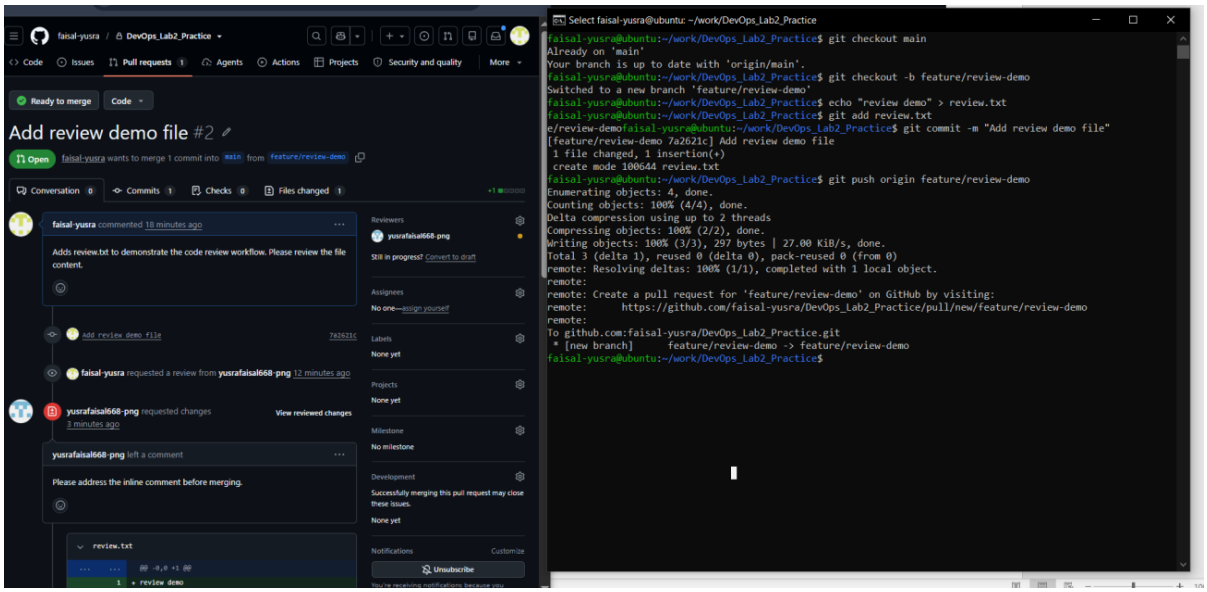
Evidence



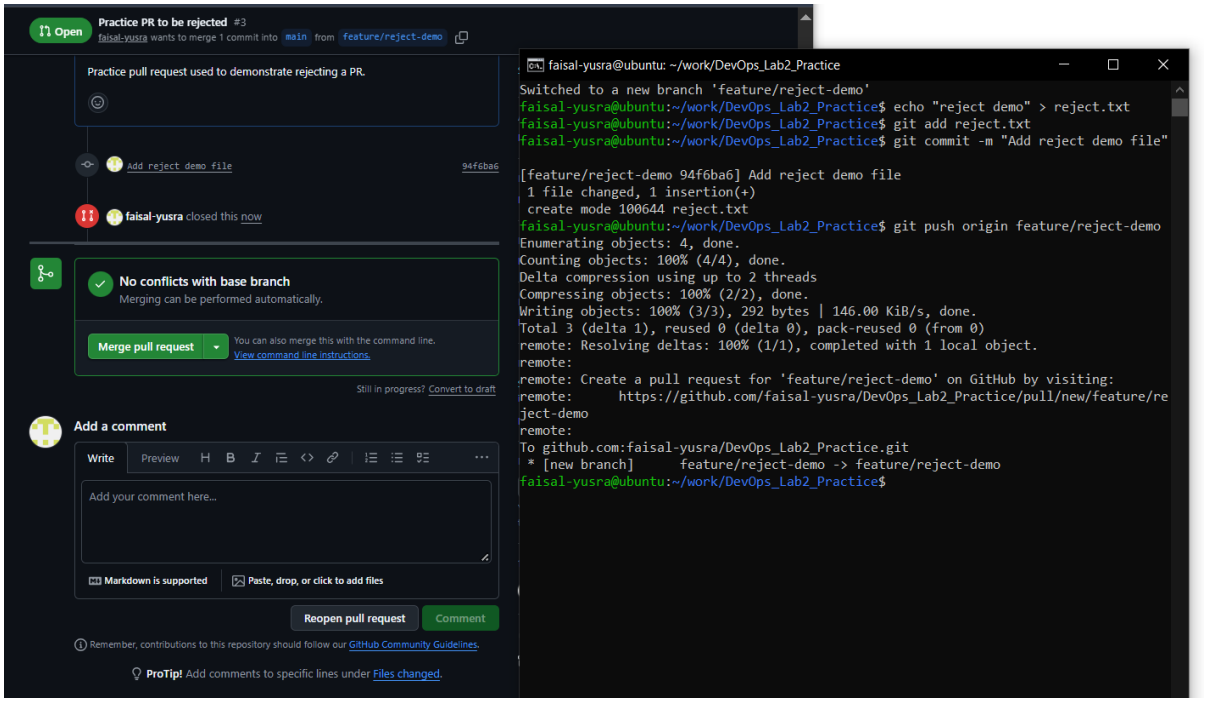
pr_create_details.png



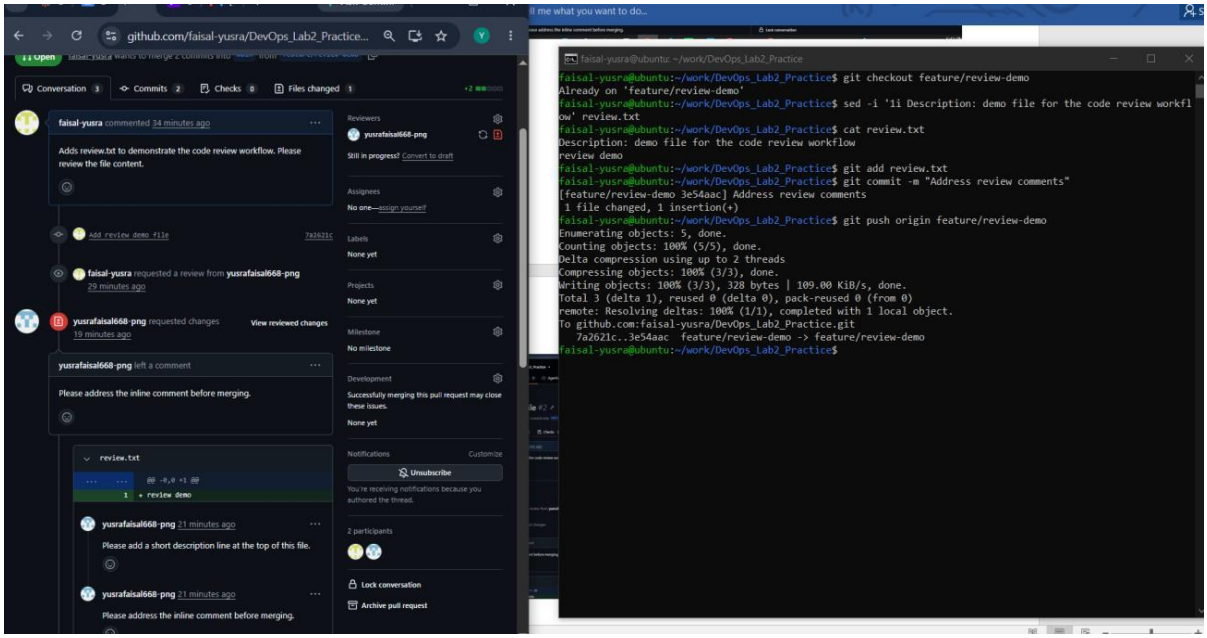
pr_assigned_reviewer.png



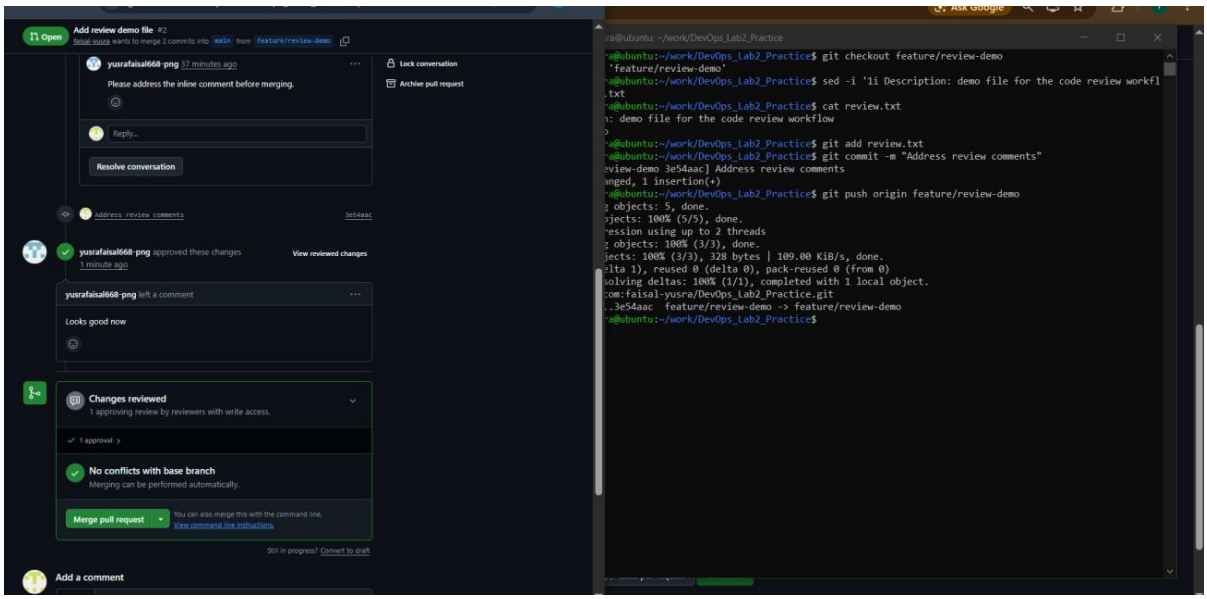
pr_request_changes.png



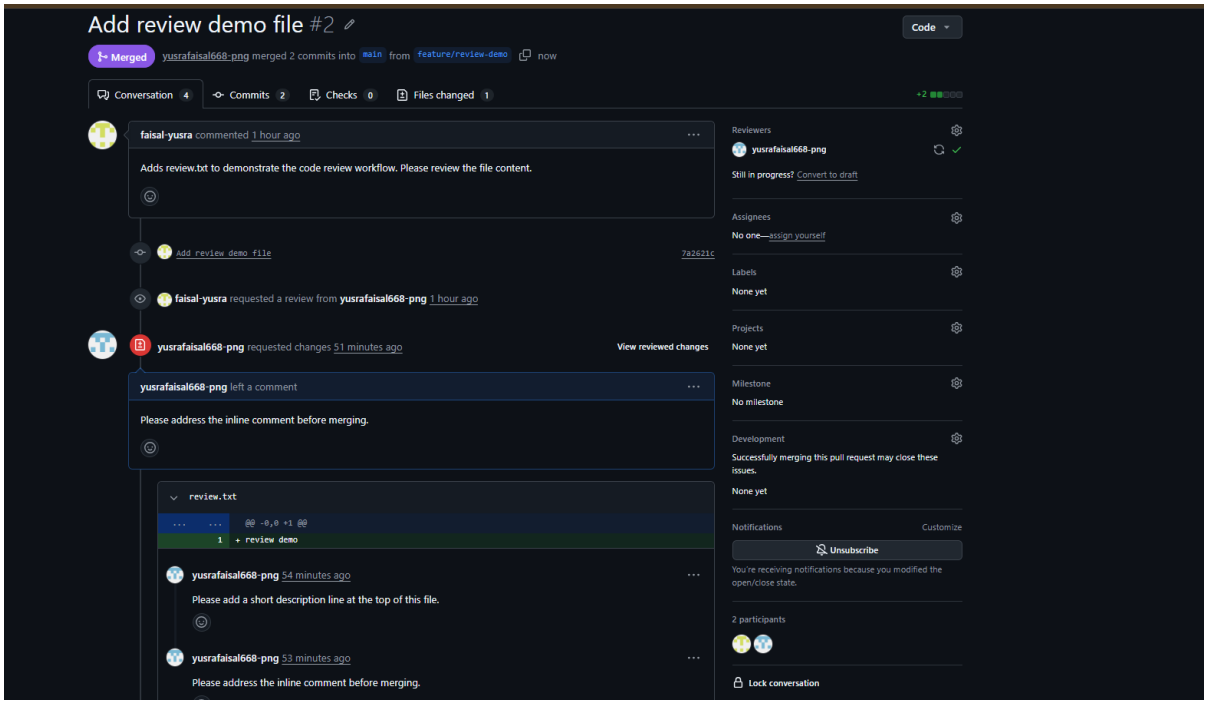
pr_rejected.png



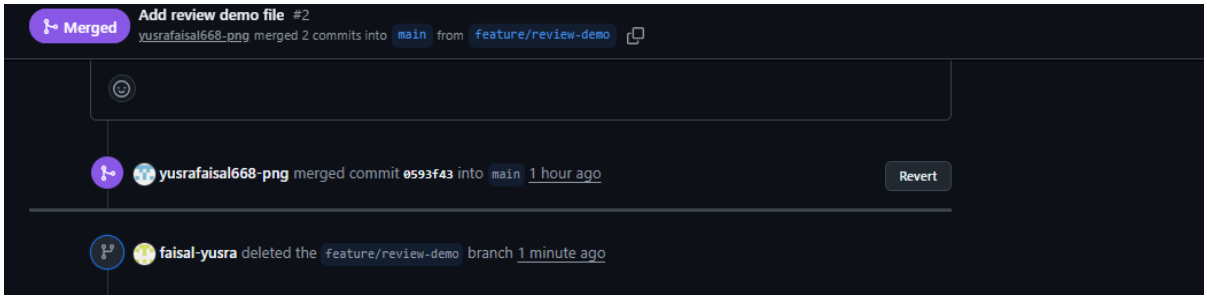
pr_updated_with_commits.png



pr_merge_confirm.png



pr_merged.png



pr_branch_deleted.png

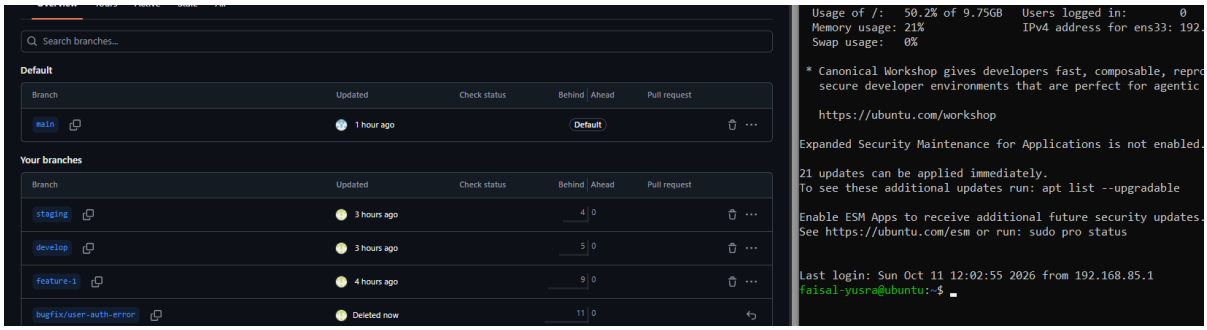
Task 13: Branch Cleanup Best Practices

One merged remote branch was deleted through the GitHub UI and another through the terminal; local references were then cleaned.

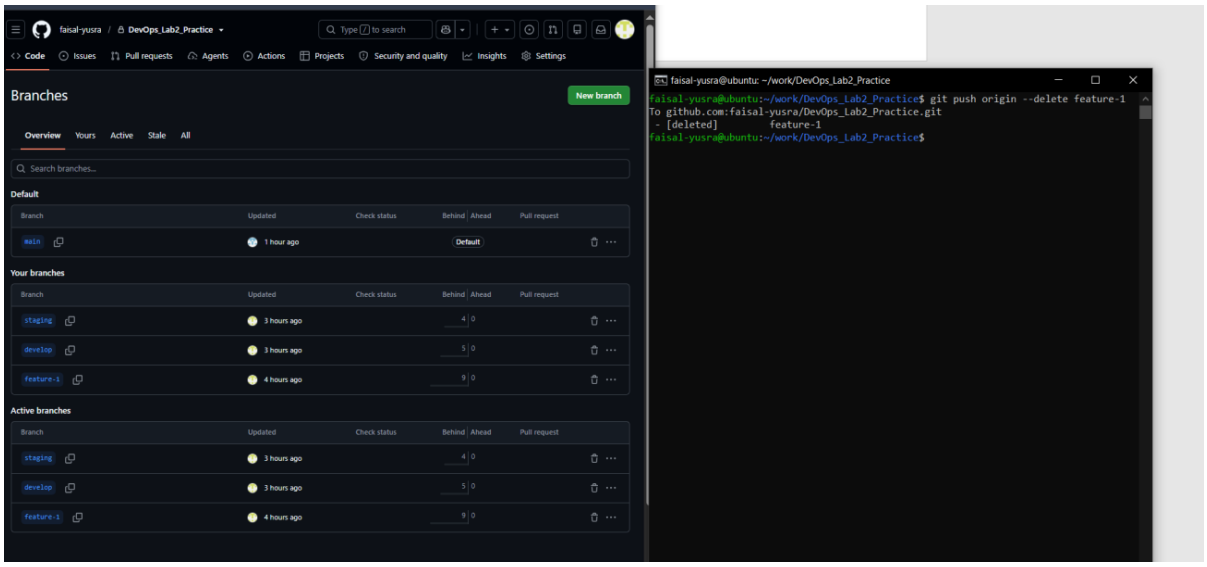
Commands

```
git push origin --delete feature-1
git checkout main
git pull origin main
git fetch --prune
git branch -d <branch-name>
```

Evidence



remote_branch_deleted.png



remote_branch_delete_cmd.png

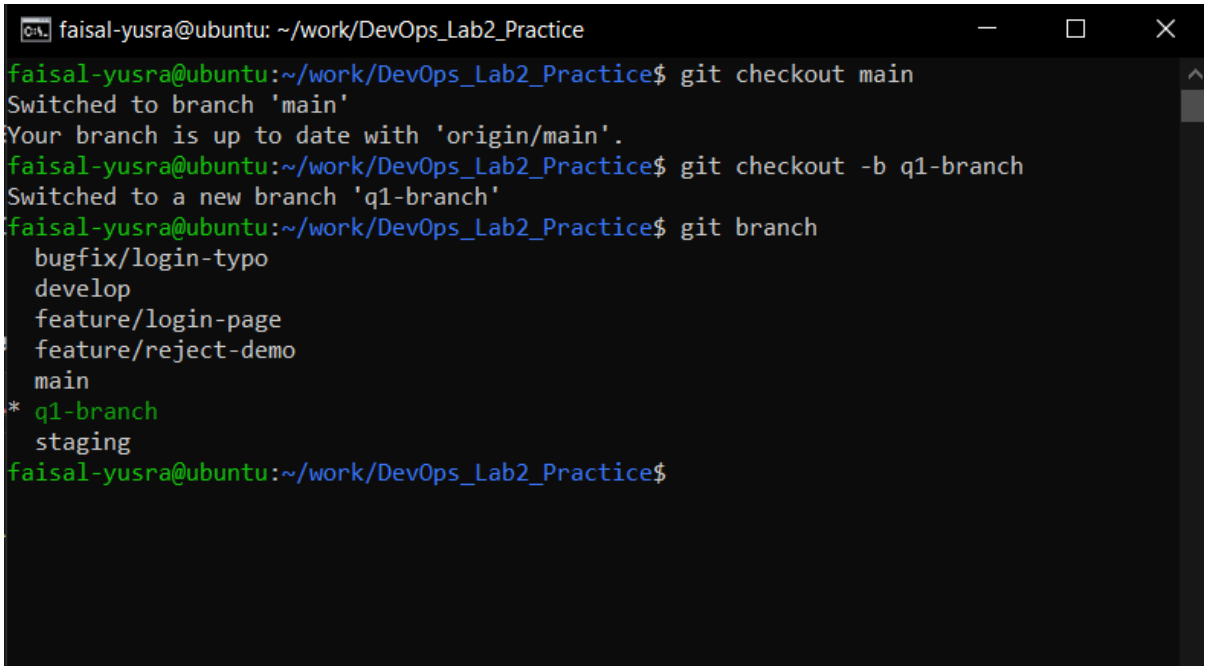
Exam Q1: Advanced Branching and Merge Verification

A branch was created, a change committed, merged into main with --no-ff, and the history graph shows the merge.

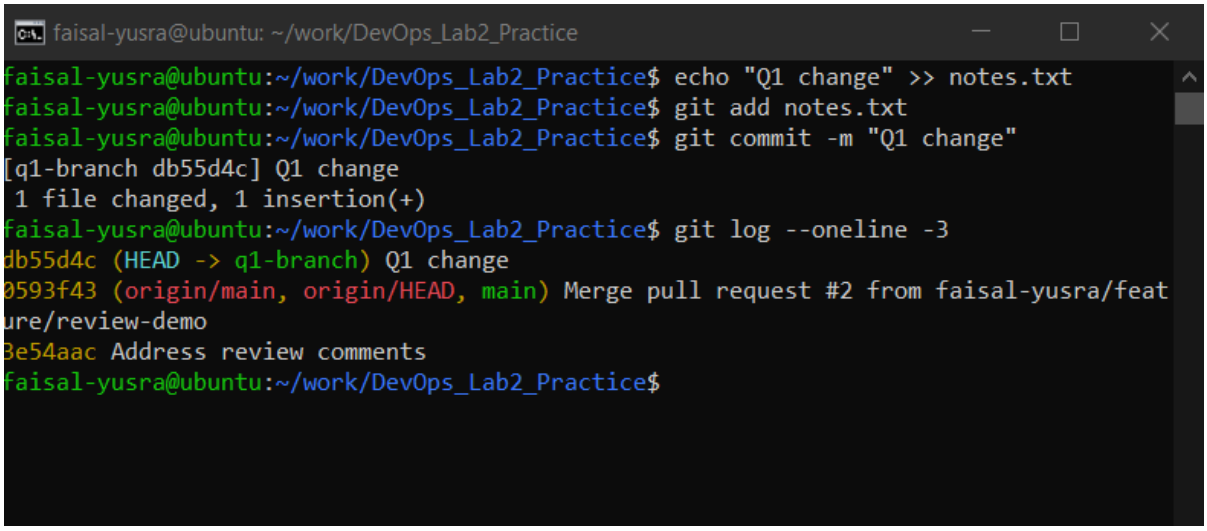
Commands

```
git checkout -b q1-branch
git commit -m "Q1 change"
git merge --no-ff q1-branch
git log --oneline --graph --all
```

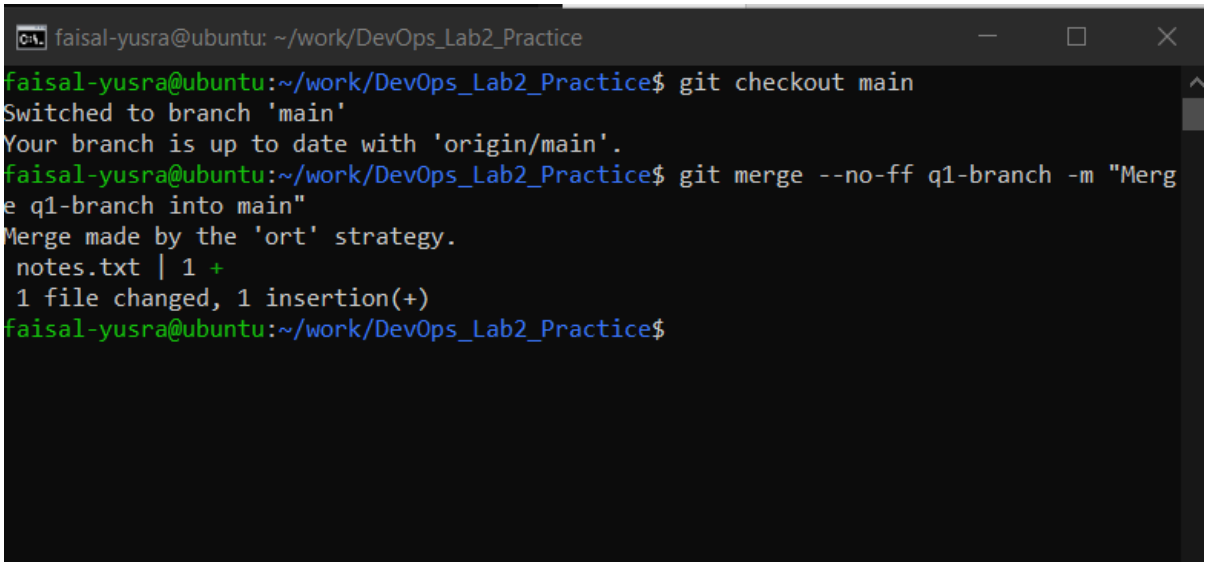
Evidence



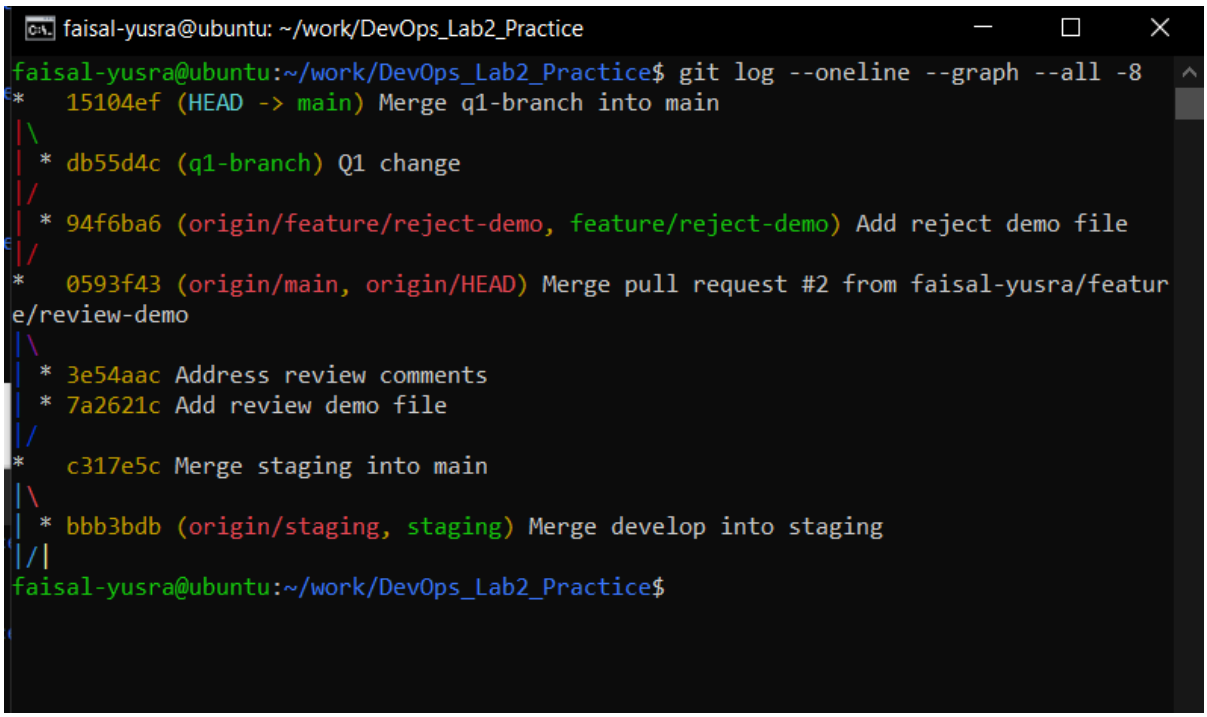
Q1_branch_created.png



Q1_commit_done.png



Q1_merge_done.png



Q1_merge_history.png

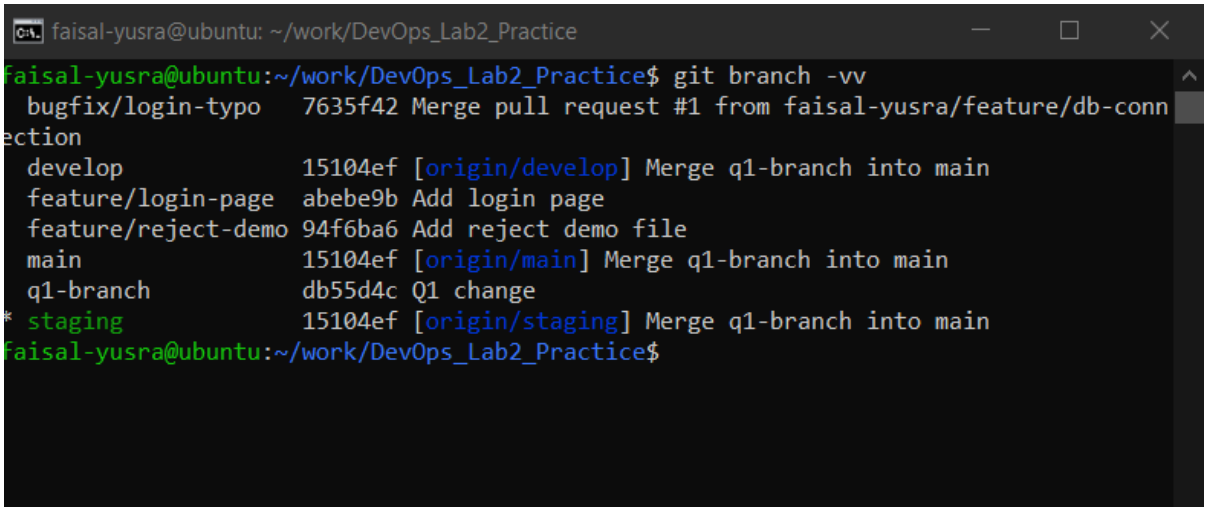
Exam Q2: Multi-Stage Workflow Simulation

main, develop and staging were aligned, a feature branch was merged into develop, then staging, then main, and git show proves each stage has the change.

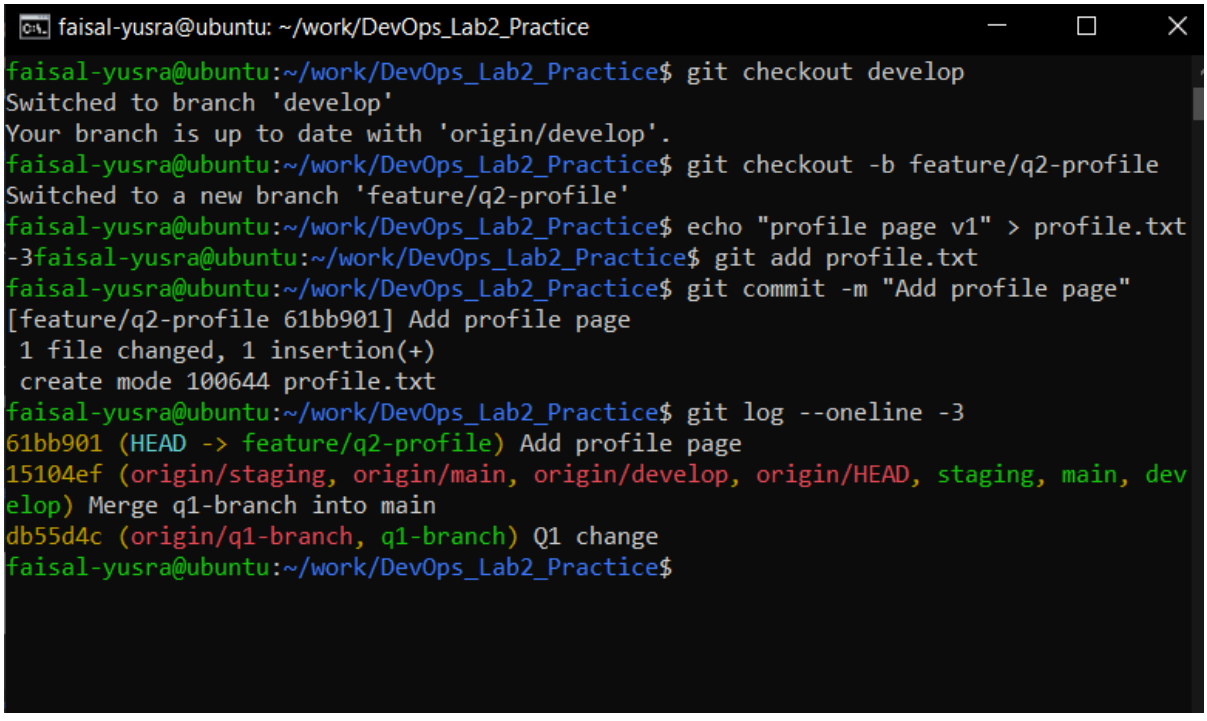
Commands

```
git checkout -b feature/q2-profile
git merge --no-ff feature/q2-profile   (develop)
git merge --no-ff develop   (staging)
git merge --no-ff staging   (main)
git show develop:profile.txt
git show staging:profile.txt
git show main:profile.txt
```

Evidence



Q2_branches_created.png



Q2_feature_branch_commit.png

```
faisal-yusra@ubuntu: ~/work/DevOps_Lab2_Practice
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git merge --no-ff feature/q2-profile
 -m "Merge feature/q2-profile into develop"
Merge made by the 'ort' strategy.
 profile.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 profile.txt
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$
```

Q2_merge_into_develop.png

```
faisal-yusra@ubuntu: ~/work/DevOps_Lab2_Practice
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git checkout staging
Switched to branch 'staging'
Your branch is up to date with 'origin/staging'.
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git merge --no-ff develop -m "Merge
develop into staging"
Merge made by the 'ort' strategy.
 profile.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 profile.txt
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$
```

Q2_merge_into_staging.png

```
faisal-yusra@ubuntu: ~/work/DevOps_Lab2_Practice
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git log --merges --oneline -3 main
64b8f09 Merge staging into main
c3d2c25 (origin/staging, staging) Merge develop into staging
d1f6972 (origin/develop, develop) Merge feature/q2-profile into develop
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git show --stat 64b8f09
commit 64b8f0923a85eed7430b814cf3813bfc31e44f20
Merge: 15104ef c3d2c25
Author: Yusra Faisal <23-20911-099@cs.fjwu.edu.pk>
Date:   Sun Oct 11 15:38:58 2026 +0000

    Merge staging into main

 profile.txt | 1 +
 1 file changed, 1 insertion(+)
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$
```

Q2_merge_into_main.png

```
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git show develop:profile.txt
profile page v1
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git show staging:profile.txt
profile page v1
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git show main:profile.txt
profile page v1
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ git log --oneline --graph --all -6
*   64b8f09 (HEAD -> main) Merge staging into main
|\
| * c3d2c25 (staging) Merge develop into staging
|/|
| * d1f6972 (develop) Merge feature/q2-profile into develop
|/|
| * 61bb901 (feature/q2-profile) Add profile page
|/
*   15104ef (origin/staging, origin/main, origin/develop, origin/HEAD) Merge q1-branch i
nto main
|\
| * db55d4c (origin/q1-branch, q1-branch) Q1 change
|/
faisal-yusra@ubuntu:~/work/DevOps_Lab2_Practice$ x
```

Q2_stage_verification.png

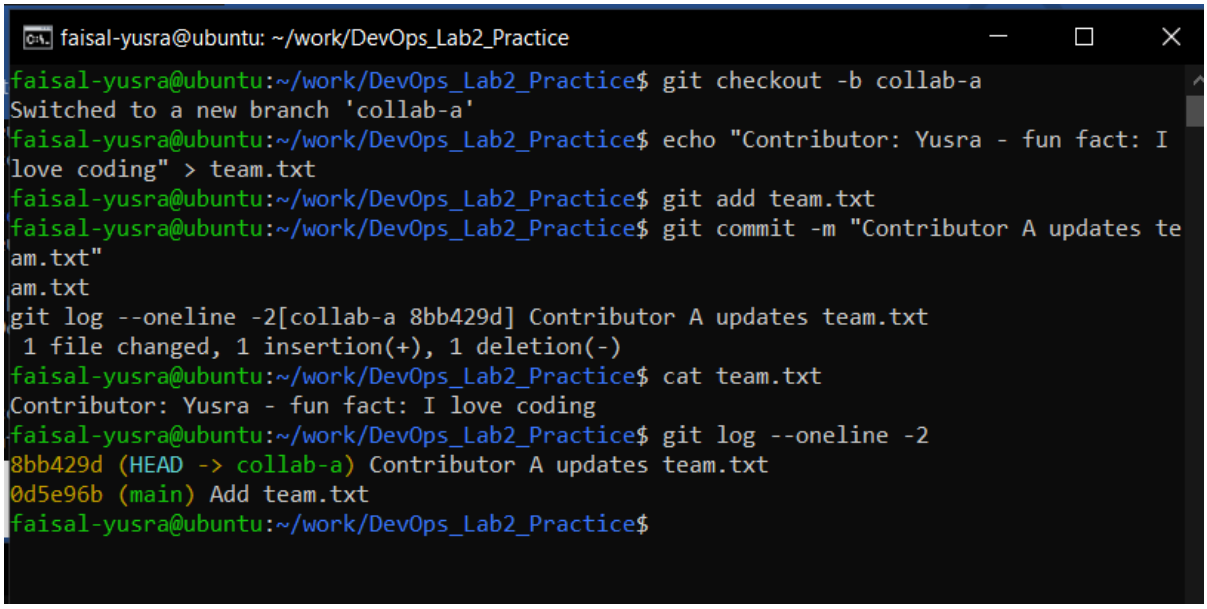
Exam Q3: Collaboration and Conflict Resolution

collab-a and collab-b changed the same line of team.txt. The merge produced a conflict, which was resolved by keeping both contributions.

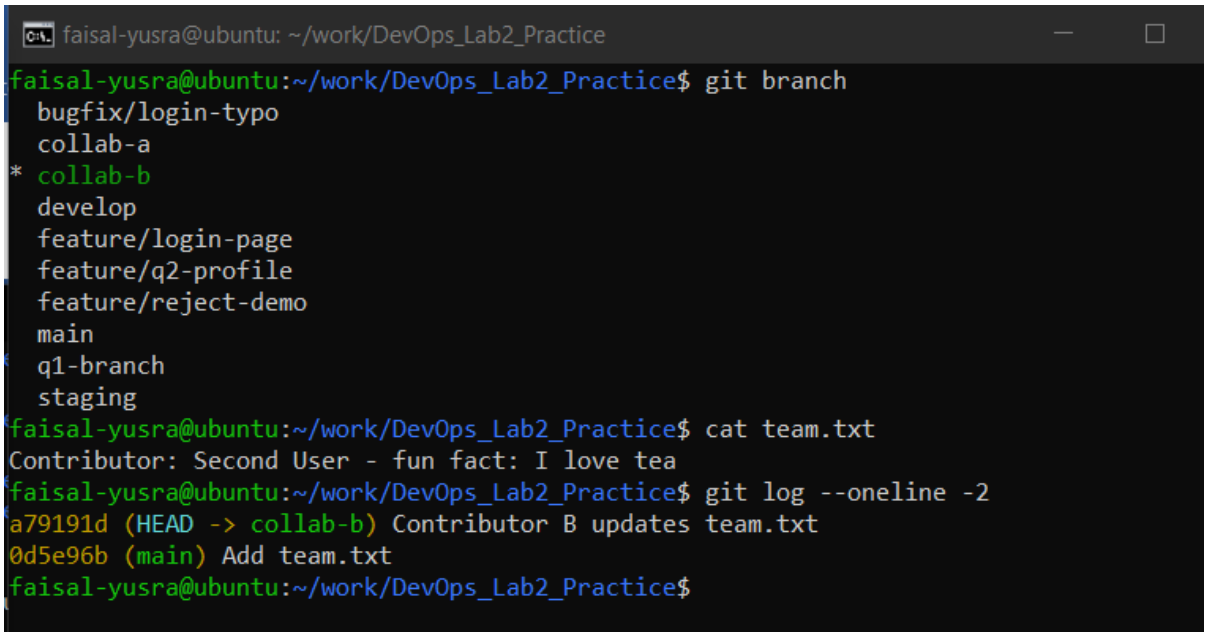
Commands

```
git merge collab-b   (CONFLICT in team.txt)
nano team.txt   (keep both lines, remove markers)
git add team.txt
git commit -m "Resolve conflict keeping both contributions"
```

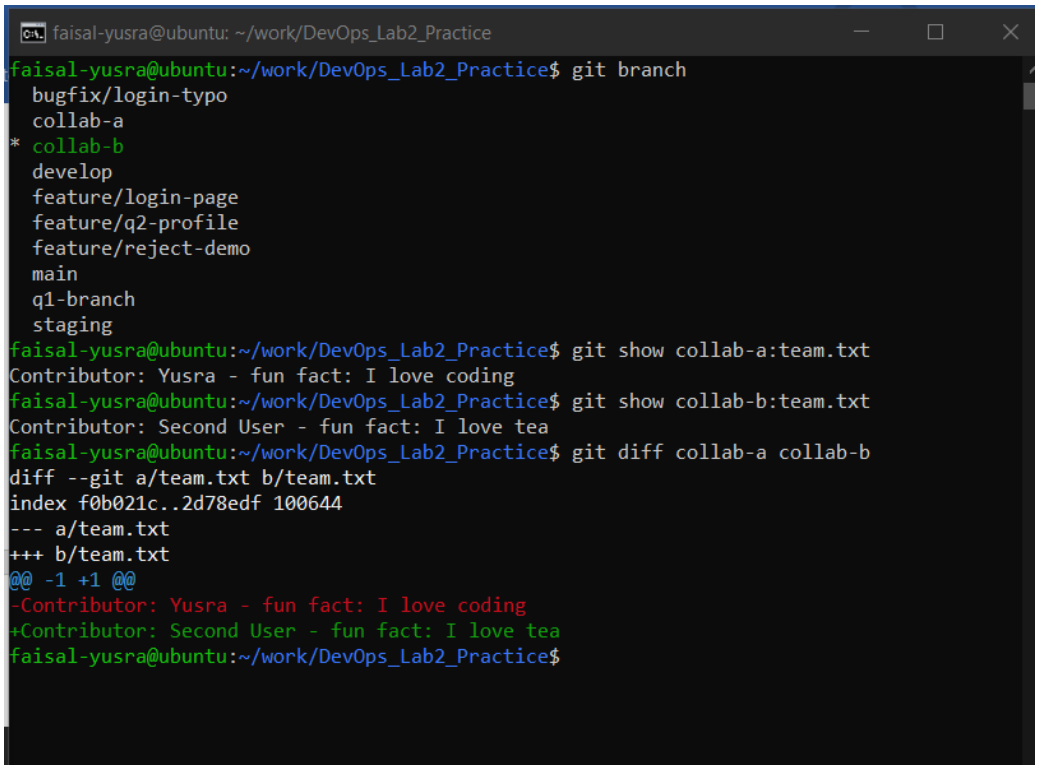
Evidence



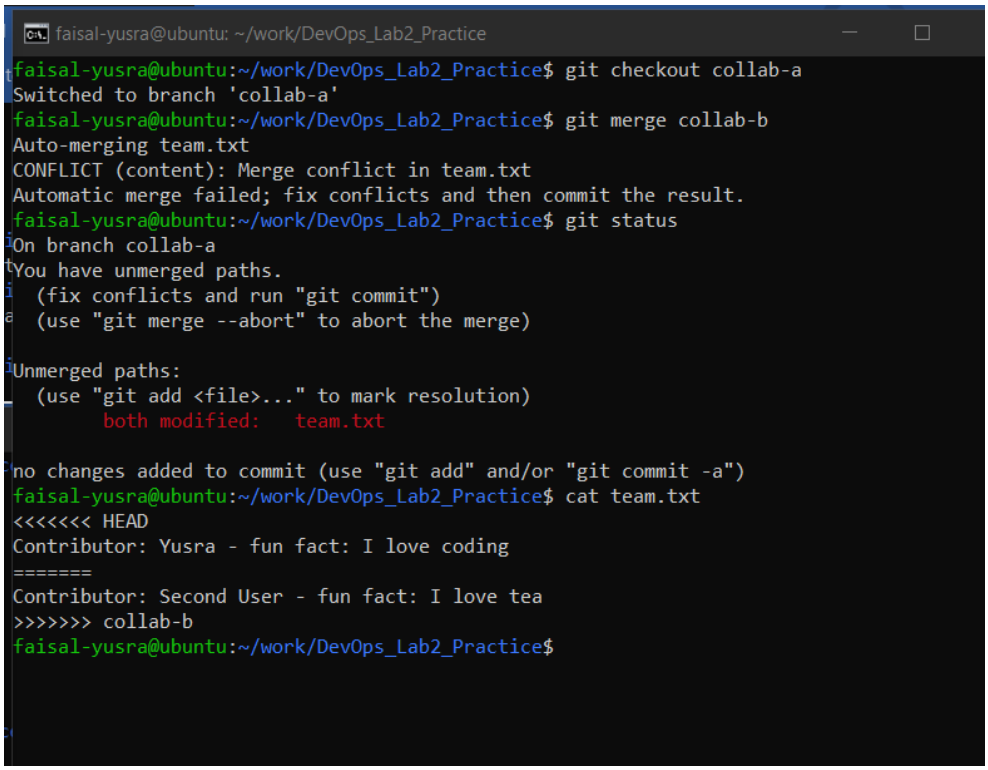
Q3_branches_modified1.png



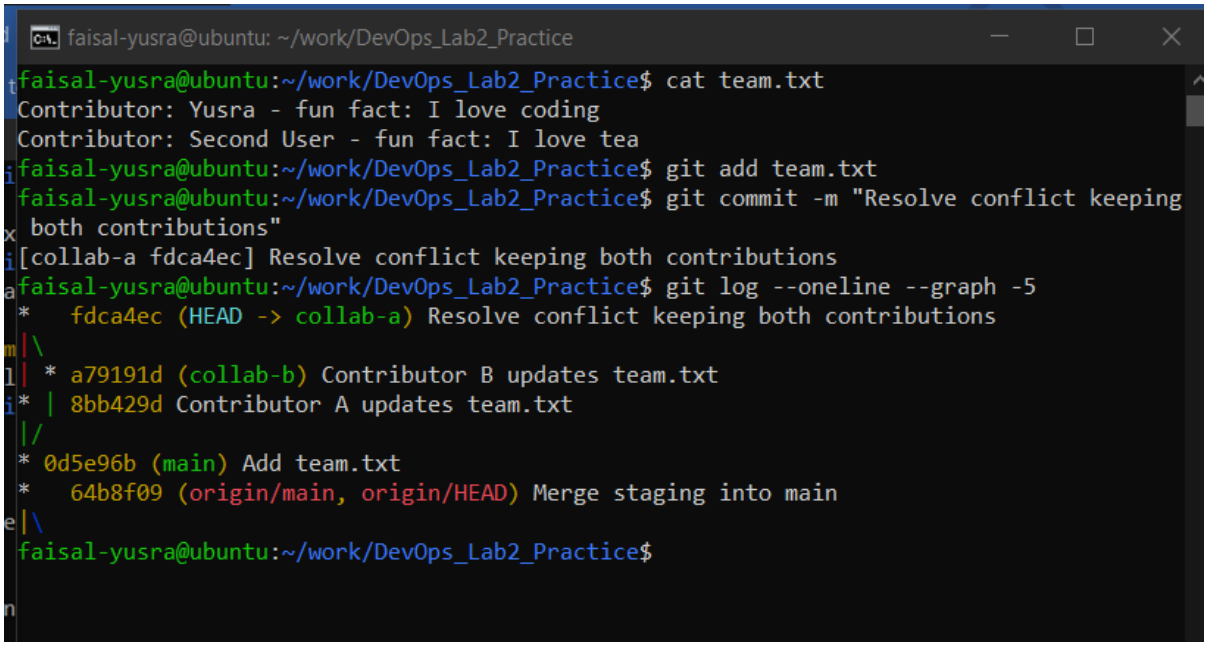
Q3_branches_modified2.png



Q3_branches_modified3.png



Q3_merge_conflict.png



Q3_conflict_resolved.png